

ParIO Runtime Workbook

This note is a guided reading of the current Parapet effect runtime.

It is meant to be read side by side with these files:

- `ParIO.scala`
- `Effect.scala`
- `ParIORuntime.scala`
- `ParIOApp.scala`
- `DslInterpreter.scala`

The style here is intentional:

- brief explanation of the function first
- then how the implementation works
- then a tiny example or mental model

It should print cleanly on A4 because it avoids wide tables and large code listings.

1. The three main ideas

`ParIO[A]`

`ParIO[A]` is the program data type.

It does not run anything by itself. It only describes a computation.

Examples:

```
ParIO.pure(42)
ParIO.delay(println("hello"))
ParIO.blocking(scala.io.Source.fromFile("data.txt").mkString)
```

Internally, `ParIO` is a small algebra of nodes:

- `Pure`
- `Delay`
- `Blocking`
- `Suspend`
- `FlatMap`
- `HandleError`
- `Sleep`

So when we write:

```
ParIO.delay(1).flatMap(n => ParIO.pure(n + 1))
```

we are building a small tree, not executing code immediately.

Effect[F]

Effect[F] is the capability interface the rest of parapet depends on.

This is the reason the scheduler and DSL interpreter can stay generic in **F[_]**.

If some effect type has:

- **given Effect[F]**
- **given Parallel[F]**

then the core runtime can use it.

That means:

- **ParIO** is one concrete effect
- **Effect** is the abstraction boundary

ParIORuntime

ParIORuntime is the interpreter for **ParIO**.

This is where the operational behavior lives:

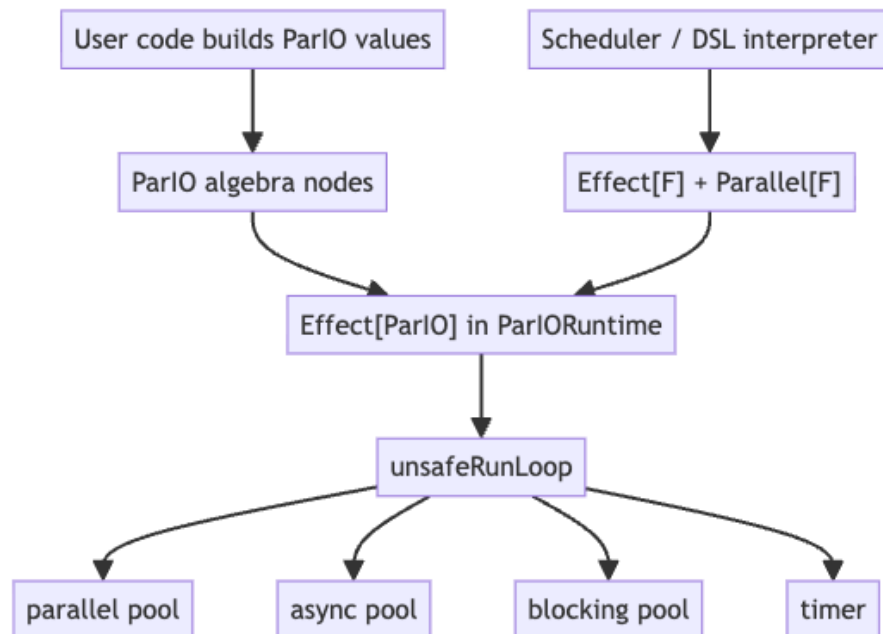
- the runloop
- thread pools
- fibers
- race
- parallel execution
- sleep
- blocking lane shift

So the split is:

- **ParIO** = description
- **ParIORuntime** = execution

That split is one of the main cleanup wins in the current version.

2. One-page architecture



What matters most:

- core parapet code talks to `Effect[F]`
 - `ParIORuntime` is just one implementation of that contract
-

3. Reading `ParIO.scala`

The important thing to see in `ParIO.scala` is that it is small again.

The sealed trait:

```
sealed trait ParIO[+A]
```

and its node types are the abstract syntax tree of the effect.

Why `map` and `flatMap` are cheap

`map` and `flatMap` do not execute. They just build new nodes:

```
final def flatMap[B](f: A => ParIO[B]): ParIO[B] =  
  ParIO.FlatMap(this, f)
```

So:

```
val p =
  ParIO.pure(10)
    .flatMap(n => ParIO.pure(n + 1))
```

means:

- keep the source program
- keep the continuation function
- let the runtime apply it later

That is exactly why we need a runloop.

What each node means

- `Pure(value)`
A value already known.
 - `Delay(thunk)`
A pure but lazy thunk.
 - `Blocking(thunk)`
A thunk that must run on the blocking lane.
 - `Suspend(thunk)`
A delayed constructor of another `ParIO`.
 - `FlatMap(source, bind)`
Run `source`, then continue with `bind`.
 - `HandleError(source, handler)`
Run `source`, and if it throws, continue with `handler`.
 - `Sleep(duration)`
Suspend until the timer fires.
-

4. Reading `Effect.scala`

The important mindset for `Effect.scala` is:

This is not “what `ParIO` can do”.
This is “what parapet core needs from any effect runtime”.

That is why it extends `Monad[F]` and adds the extra runtime operations:

- `delay`
- `blocking`
- `suspend`
- `raiseError`
- `sleep`
- `start`

- `startBlocking`
- `race`
- `guarantee`

Why `startBlocking` exists

This is one of the key runtime changes.

Before, ordinary background work and offloaded blocking work were too close together.

Now there is a deliberate split:

- `start(fa)`
Run `fa` on the async lane.
- `startBlocking(fa)`
Run `fa` on the blocking lane.

That lets the DSL mean something more honest:

- `fork` uses `start`
- `offload` uses `startBlocking`

What `EffectFiber` means

`EffectFiber[F, A]` is a handle to a running effectful computation:

- `join: F[A]`
- `cancel: F[Unit]`

So a fiber is not the computation itself. It is the handle used to observe or stop that computation.

5. Reading `ParIORuntime.scala`

`ParIORuntime.scala` is the most important file now.

Read it in this order:

1. config types
2. lane model
3. effect instance
4. `unsafeRunLoop`
5. fiber helpers
6. race/parallel helpers
7. blocking/timer helpers

6. Runtime configuration and lanes

ParIORuntime owns four executors:

- parallelPool
- asyncPool
- blockingPool
- timer

They correspond to four logical roles:

- scheduler worker lane
- background async lane
- blocking/offload lane
- wakeup lane

Why a Lane enum exists

The runtime has:

```
private enum Lane:  
  case External, Parallel, Async, Blocking
```

and a thread-local:

```
private val laneLocal = new ThreadLocal[Lane]()
```

This is a small but important trick.

It lets the runtime know:

- where a computation is currently running
- whether it already is on the blocking lane

That is used by `runBlocking`:

- if already on `Lane.Blocking`, run inline
- otherwise shift to the blocking pool

Without that check, blocking work could re-submit itself forever or create useless extra handoffs.

7. The runloop: unsafeRunLoop

This is the heart of the runtime.

Brief explanation

`unsafeRunLoop` is the interpreter for `ParIO`.

Its job is:

1. inspect the current `ParIO` node
2. decide what to do next
3. keep track of pending continuations
4. return a value or throw an uncaught error

The two core variables

The center of the method is:

```
var current: ParIO[Any] = io.asInstanceOf[ParIO[Any]]
var stack: List[Frame] = Nil
```

These two variables are the whole trampoline.

What `current` means

`current` is the node being interpreted right now.

Examples:

- `Pure(42)`
- `Delay(() => 1 + 1)`
- `FlatMap(source, bind)`

At each loop iteration, the runtime pattern-matches on `current`.

What `stack` means

`stack` is a heap-allocated continuation stack.

It is not the JVM call stack.

That difference is the key to stack safety.

Instead of recursive calls like:

```
run(source)
run(bind(result))
run(next)
```

the runtime stores “what to do next” in `stack` and keeps looping.

What a `Frame` is

A `Frame` is one suspended continuation.

Current frame types:

- `BindFrame(run)`
- `RecoverFrame(run)`

Meaning:

- `BindFrame` = “when you get a value, continue with this `flatMap` function”

- RecoverFrame = “if you get an error, continue with this recovery function”

Why this is stack-safe

Suppose we write:

```
def loop(n: Int): ParIO[Int] =
  if n <= 0 then ParIO.pure(0)
  else ParIO.pure(()).flatMap(_ => loop(n - 1))
```

This can build a very deep chain of FlatMap nodes.

If the runtime used plain recursion, the JVM call stack would grow with every bind.

Instead, `unsafeRunLoop`:

- pushes continuations into `stack`
- stays inside one `while true` loop
- uses constant JVM stack depth

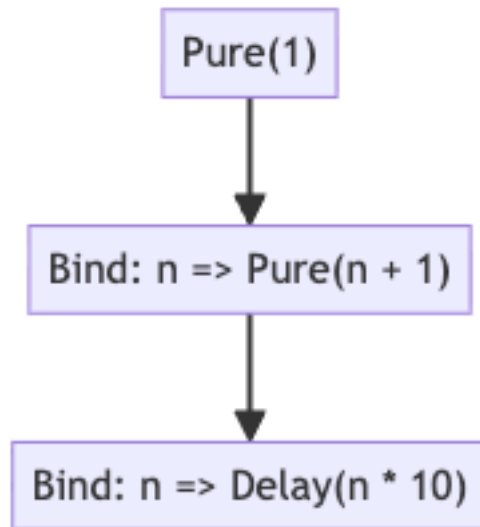
That is the classic trampoline idea.

8. Walking through a tiny example

Take this program:

```
val io =
  ParIO.pure(1)
    .flatMap(n => ParIO.pure(n + 1))
    .flatMap(n => ParIO.delay(n * 10))
```

We can think of it as:



Step-by-step

1. `current = FlatMap(FlatMap(Pure(1), f1), f2)`
2. `push BindFrame(f2)`
3. move into the source `FlatMap(Pure(1), f1)`
4. `push BindFrame(f1)`
5. move into `Pure(1)`
6. `pop BindFrame(f1)` and compute `Pure(2)`
7. `pop BindFrame(f2)` and compute `Delay(() => 20)`
8. evaluate `Delay`
9. end with `Pure(20)` and empty stack
10. return 20

No deep recursive JVM call chain is needed.

9. Reading the `unsafeRunLoop` cases

`Pure(value)`

Briefly:

- if there is no pending continuation, we are done
- otherwise feed the value to the next continuation

Implementation idea:

```
case Pure(value) =>
  stack match
```

```

case Nil => return value
case BindFrame(run) :: tail =>
  current = run(value)
  stack = tail
case RecoverFrame(_) :: tail =>
  current = Pure(value)
  stack = tail

```

Why `RecoverFrame(_)` is skipped on success:

- error handlers matter only on failure
- a successful value should pass through them untouched

Delay(thunk)

Briefly:

- evaluate the thunk now
- wrap the result as `Pure`

Implementation:

```

case Delay(thunk) =>
  current = Pure(thunk())

```

The important subtlety:

- exceptions thrown by `thunk()` are caught by the outer `try/catch`
- so `handleErrorWith` still works

Blocking(thunk)

Briefly:

- run the thunk through the blocking lane helper
- then continue as `Pure(result)`

Implementation:

```

case Blocking(thunk) =>
  current = Pure(runBlocking(thunk()))

```

The lane logic is not inside the algebra node. It is inside the runtime helper.

Suspend(thunk)

Briefly:

- force a delayed `ParIO`
- continue with that program

Implementation:

```
case Suspend(thunk) =>
  current = thunk()
```

This is important when you want laziness at the level of building the next `ParIO`.

FlatMap(source, bind)

Briefly:

- do not run `bind` yet
- first remember it as a continuation
- then move into `source`

Implementation:

```
case FlatMap(source, bind) =>
  current = source
  stack = BindFrame(bind) :: stack
```

This is the single most important stack-safety move in the whole runtime.

HandleError(source, handler)

Briefly:

- do not run `handler` yet
- first remember it as an error continuation
- then move into `source`

Implementation:

```
case HandleError(source, handler) =>
  current = source
  stack = RecoverFrame(handler) :: stack
```

Sleep(duration)

Briefly:

- wait until a timer fires
- then continue with `Pure(())`

Implementation:

```
case Sleep(duration) =>
  sleepOnTimer(duration)
  current = Pure(())
```

This is cleaner than `Thread.sleep`, but still not fully asynchronous, because the runloop waits synchronously.

10. How error handling works

If any node throws, the outer `catch` runs.

The runtime then scans `stack` until it finds a `RecoverFrame`.

Brief explanation

Success uses `BindFrame`.

Failure uses `RecoverFrame`.

So the continuation stack stores both:

- normal “what next”
- error “what next”

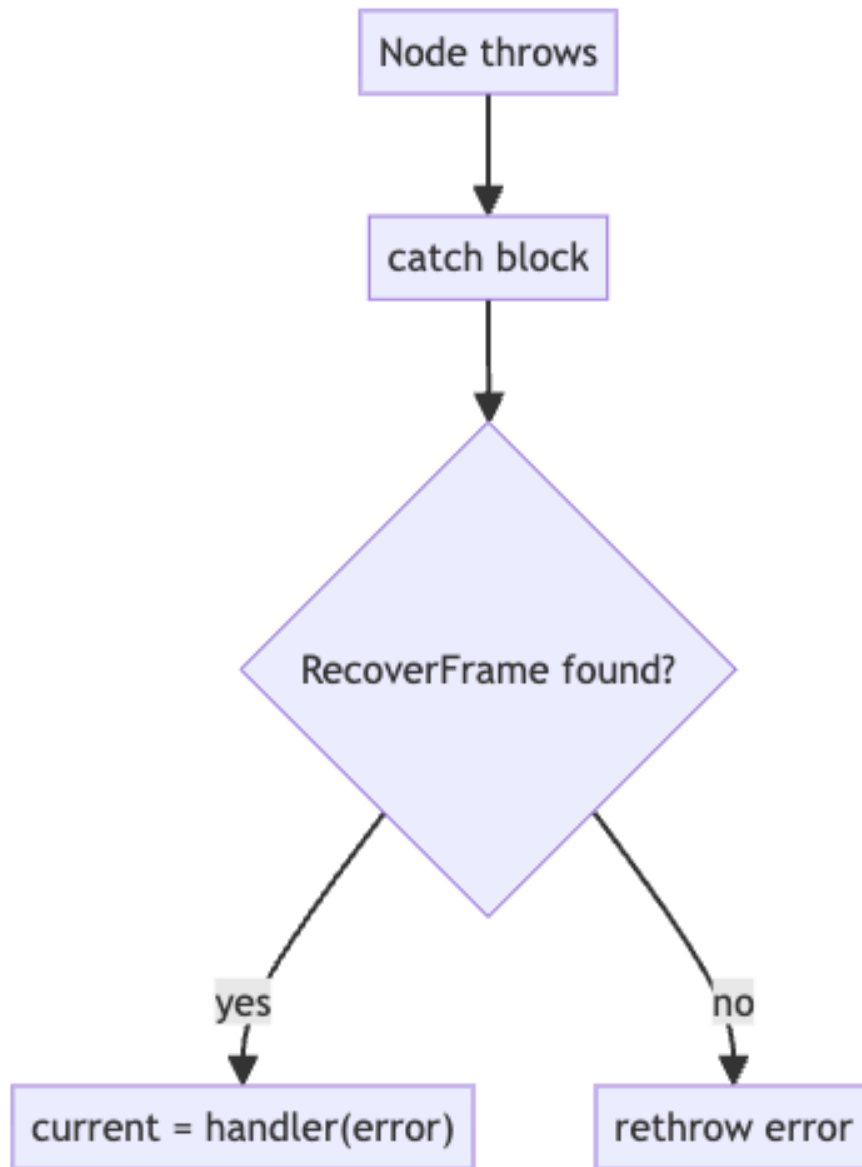
Example

```
ParIO.raiseError[Int](new RuntimeException("boom"))  
  .handleErrorWith(_ => ParIO.pure(123))
```

Rough flow:

1. push `RecoverFrame`
2. enter the source
3. source throws
4. catch block scans the stack
5. finds `RecoverFrame`
6. sets `current = handler(error)`
7. continues with `Pure(123)`

Flowchart:



11. `startFiberOn`

This is one of the most important helper methods.

Brief explanation

`startFiberOn` takes:

- an executor
- a lane label
- a `ParIO[A]`

and returns an `EffectFiber[ParIO, A]`.

So its job is:

1. submit a program to some executor
2. run the `ParIO` program there
3. capture its result or failure
4. return a fiber handle with `join` and `cancel`

Implementation shape

It creates:

- a `CompletableFuture[A]` called `result`
- an `AtomicReference[Thread]` called `runner`
- a submitted executor task

The executor task does:

1. remember the running thread in `runner`
2. set the lane with `runOnLane`
3. call `unsafeRunLoop(fa)`
4. complete `result` with success or failure

That means the fiber handle does not store the result directly. It stores a future that will eventually contain it.

Why `join` is implemented with `ParIO.blocking`

`join` is:

```
def join: ParIO[A] =  
  ParIO.blocking(await(result))
```

That means:

- joining a running fiber is a potentially blocking wait
- so it is forced onto the blocking lane

This is a good and honest choice for the current runtime.

Why `cancel` interrupts the runner thread

The runtime does not yet have sophisticated fiber cancellation semantics.

So `cancel` is best-effort:

1. interrupt the running thread if known
2. cancel the executor task
3. wait a bit for the result future to settle

That is not as strong as Cats Effect cancellation, but it matches the current implementation model.

Tiny example

If we call:

```
val fiber = runtime.unsafeRun(runtime.effect.start(ParIO.delay(1 + 1)))
val value = runtime.unsafeRun(fiber.join)
```

then:

1. `start` uses `startFiberOn(asyncPool, Lane.Async, ...)`
 2. a background task runs `unsafeRunLoop`
 3. the result future is completed with 2
 4. `join` waits for that future on the blocking lane
-

12. racePrograms

Brief explanation

`racePrograms(left, right)` runs two programs concurrently and returns whichever finishes first.

Result type:

`Either[A, B]`

Meaning:

- `Left(a)` means the left side won
- `Right(b)` means the right side won

Implementation idea

It uses `ExecutorCompletionService`.

That class is useful because it lets us:

- submit several tasks
- then wait for the first completed one

The method:

1. submits left and right onto the async pool
2. tags them as 0 or 1
3. waits for the first completed result

4. cancels the loser
5. returns the winner

Why tagging is needed

Both sides eventually produce a value of some unknown type.

So the helper wraps each completion as:

```
RaceResult(tag: Int, value: Any)
```

The tag tells us which side completed.

Failure behavior

If one side finishes by throwing:

- `completion.take().get()` throws `ExecutionException`
- the method unwraps `getCause`
- the other side is cancelled
- the underlying cause is rethrown

That keeps error reporting direct.

Example

```
runtime.unsafeRun(  
  runtime.effect.race(  
    ParIO.sleep(1.second).flatMap(_ => ParIO.pure("slow")),  
    ParIO.pure("fast")  
  )  
)
```

Expected result:

```
Right("fast")
```

because the right side finishes first.

13. runParallel

Brief explanation

`runParallel` is the implementation of `Parallel[ParIO].par`.

Its job is:

1. run many `ParIO[Unit]` effects on the parallel pool
2. wait for all of them
3. fail fast if one fails
4. cancel the rest on failure

Why this is not the same as `race`

`race` cares about the first finisher.

`runParallel` cares about:

- all successes, or
- the first observed failure

So it uses the same `ExecutorCompletionService` idea, but with a different loop.

Implementation

1. submit every effect to `parallelPool`
2. store the resulting futures
3. keep a counter `remaining`
4. repeatedly call `completion.take().get()`
5. decrement `remaining` on success
6. on failure, cancel everyone and rethrow the cause
7. in `finally`, cancel every future again just to clean up

This is a good pattern for fail-fast parallel execution on top of Java executors.

Example

```
runtime.unsafeRun(  
  runtime.parallel.par(  
    Seq(  
      ParIO.delay(println("a")),  
      ParIO.delay(println("b")),  
      ParIO.delay(println("c"))  
    )  
  )  
)
```

The effects run in parallel pool threads, and the method returns only when all have completed.

If one throws, the caller sees that error directly.

14. `runBlocking`

Brief explanation

`runBlocking` ensures a thunk runs on the blocking lane.

Implementation rule

```
Option(laneLocal.get()) match
  case Some(Lane.Blocking) => thunk
  case _                    => await(submitThunk(blockingPool, Lane.Blocking)(thunk))
```

So:

- already on blocking lane -> run inline
- otherwise -> submit and wait

Why this matters

This avoids unnecessary re-submission when blocking code itself calls another blocking operation.

Without that check:

- blocking work could bounce between threads for no reason
 - nesting `blocking` would be much noisier
-

15. `sleepOnTimer`

Brief explanation

`sleepOnTimer` implements `ParIO.Sleep`.

It uses:

- the timer executor to schedule a wakeup
- a `CompletableFuture[Unit]` as the wakeup signal
- `runBlocking` to wait for that signal on the blocking lane

How it works

1. create `signal`
2. schedule `signal.complete()` after `duration`
3. wait for `signal`
4. cancel the scheduled task if needed

This is better than raw `Thread.sleep`, because:

- timekeeping is handled by the timer executor
- the waiting happens on the blocking lane, not inline on whatever lane called `sleep`

But it is still synchronous from the runloop's point of view.

That is one of the main remaining limitations.

16. `runOnLane` and `await`

These methods are small, but they explain a lot of runtime behavior.

`runOnLane`

Briefly:

- set the thread-local lane
- run the thunk
- restore the previous lane

This is how nested runtime calls preserve context correctly.

`await`

Briefly:

- wait for a `Java Future`
- unwrap `ExecutionException` into the real cause
- re-interrupt the current thread on `InterruptedException`

That is the bridge between Java executor APIs and the effect runtime.

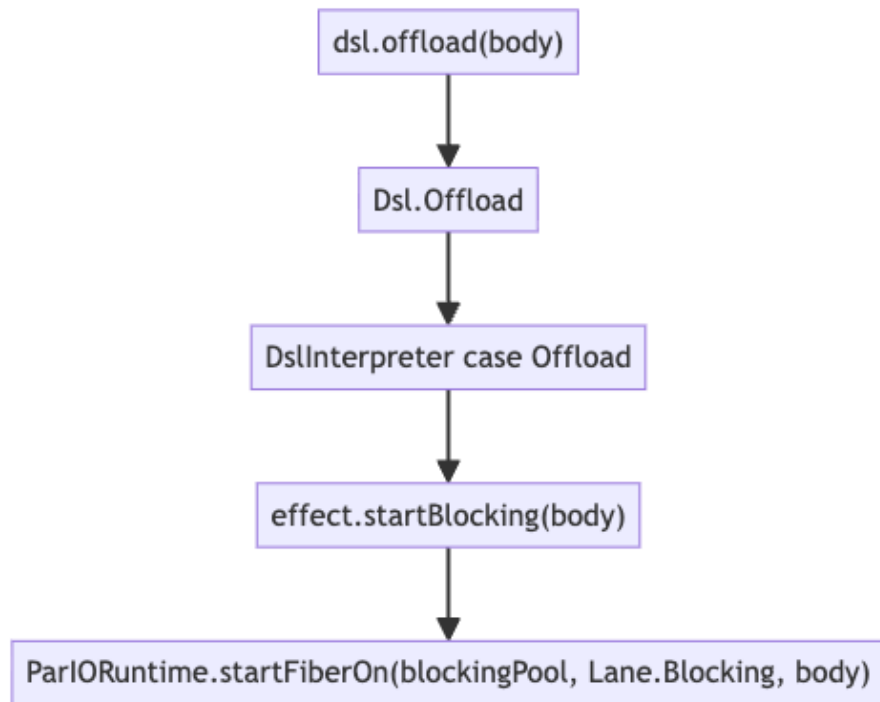
17. How `offload` reaches the runtime

This is the most relevant end-to-end example for `parapet` users.

In `DslInterpreter.scala`, the `Offload` case does:

1. build the effectful body
2. start it with `effect.startBlocking(...)`
3. track completion in the process offload tracker

So the path is:



This is why the new `startBlocking` primitive matters. It gives `offload` a runtime meaning distinct from plain `fork`.

18. Review checklist

When reading the code, these are the best questions to ask.

About `ParIO`

- Is the algebra minimal but sufficient?
- Are the node names honest?
- Are `flatMap` and `handleErrorWith` represented cleanly?

About `Effect`

- Is every method truly needed by parapet core?
- Does `startBlocking` belong at this boundary?
- Are the semantics understandable for other possible runtimes?

About `ParIORuntime`

- Is the lane model clear?

- Are the pool defaults reasonable?
- Is `unsafeRunLoop` easy to reason about?
- Are `startFiberOn`, `racePrograms`, and `runParallel` correct under failure?
- Are interruption and cancellation honest about their limits?

About remaining limitations

- Where do we still synchronously wait?
 - Which operations still consume real threads while waiting?
 - What would require a true async callback node to improve further?
-

19. What would move this closer to 8/10

Without trying to become Cats Effect, the biggest next upgrades would be:

1. add a true async callback node to `ParIO`
2. make `sleep` fiber-parking instead of synchronously waiting
3. improve cancellation semantics beyond interruption-based best effort
4. revisit blocking pool defaults once waits no longer consume blocking threads the same way

Those are the biggest design moves left.

20. Sources worth studying

These are the most useful sources for understanding or validating this style of runtime.

Book

- *Functional Programming in Scala* by Chiusano and Bjarnason
Best source for the mindset behind algebraic descriptions of effects, trampolining, and interpreters.

Paper / article

- Runar Bjarnason, *Stackless Scala With Free Monads*
Very useful for understanding why the `current + stack` trampoline pattern exists at all.

Standard library concept

- Scala `TailCalls` and trampoline-style interpreters
Good for the general idea of turning recursive interpretation into iterative

stepping.

Runtime design references

- Cats Effect thread model
- Cats Effect scheduler/runtime docs
- JDK `ThreadPoolExecutor` docs

These are especially useful for:

- compute vs blocking separation
 - cancellation tradeoffs
 - timer design
 - pool sizing and rejection behavior
-

21. Final mental model

If you want one compact summary, use this:

`ParIO` is a small program tree. `Effect` is the capability boundary the parapet core depends on. `ParIORuntime` is the interpreter that walks the tree with a trampoline (`current + stack`), and when needed it hands work to one of four lanes: parallel, async, blocking, or timer.

That is the whole picture.